

Sumário

1	Relatório — Lab02S02: Qualidade de Repositórios Java Populares	1
1.1	1. Introdução	1
1.2	2. Metodologia	1
1.3	3. Hipóteses	1
1.4	4. Resultados	2
1.5	5. Discussão	4
1.6	6. Conclusão	4
1.7	7. Reprodutibilidade	5

1 Relatório — Lab02S02: Qualidade de Repositórios Java Populares

Disciplina: Laboratório de Experimentação de Software

Data: abril de 2026

1.1 1. Introdução

Este relatório analisa a qualidade interna de **1.000 repositórios Java** mais populares no GitHub, medida pelas métricas CBO, DIT e LCOM (ferramenta CK). O objetivo é verificar se características de processo — popularidade, maturidade, atividade e tamanho — correlacionam-se com a qualidade do código-fonte. As questões de pesquisa e hipóteses são investigadas com correlação de Spearman sobre o dataset coletado via REST Search API do GitHub.

1.2 2. Metodologia

Etapa	Descrição
Coleta	REST Search API GitHub — 1.000 repositórios Java por número de estrelas
Métricas CK	Clone shallow + JAR CK 0.7.0 → CBO, DIT, LCOM médios por repositório
Métricas de processo	Stars, AgeInYears, ReleasesCount, TotalLoc (contagem de linhas <code>.java</code>)
Análise	Correlação de Spearman (scipy) + estatísticas descritivas (pandas)
Dataset final	969/1.000 repositórios com CK válido; 991/1.000 com LOC

Repositórios sem CK válido (31/1.000) falharam por limitações do parser JDT/CK (`NullPointerException`, `OutOfMemoryError`) em projetos com construções Java modernas ou volume muito grande — classificados como ameaça à validade interna.

1.3 3. Hipóteses

RQ	Variável de processo	Hipótese
RQ01	Stars (popularidade)	H01: correlação negativa entre Stars e CBO/LCOM
RQ02	AgeInYears (maturidade)	H02: efeito ambíguo; possivelmente sem significância
RQ03	ReleasesCount (atividade)	H03: correlação negativa entre releases e CBO/LCOM
RQ04	TotalLoc (tamanho)	H04: correlação positiva entre LOC e CBO/LCOM

1.4 4. Resultados

1.4.1 Estatísticas descritivas (n=969)

Variável	Mediana	Média	DP
Stars	5.791	9.428	10.698
AgeInYears	10 anos	9,6	3,2
ReleasesCount	0	0	0
TotalLoc	27.353	153.600	354.498
AvgCbo	5,23	5,28	1,84
AvgDit	1,38	1,44	0,34
AvgLcom	24,36	117,27	1.757,53

1.4.2 RQ01 — Popularidade × Qualidade

Hipótese: correlação negativa entre Stars e CBO/LCOM.

Métrica	r (Spearman)	p-valor	Significância
CBO	0,026	0,427	ns
DIT	-0,054	0,093	ns
LCOM	0,058	0,069	ns

Resultado: nenhuma correlação significativa. Hipótese **refutada**.

Interpretação: popularidade no GitHub (estrelas) não implica maior qualidade interna de código. Repos com muitas estrelas incluem desde projetos bem arquitetados até tutoriais e listas de recursos com pouco código Java.

1.4.3 RQ02 — Maturidade × Qualidade

Hipótese: efeito ambíguo; possivelmente sem significância.

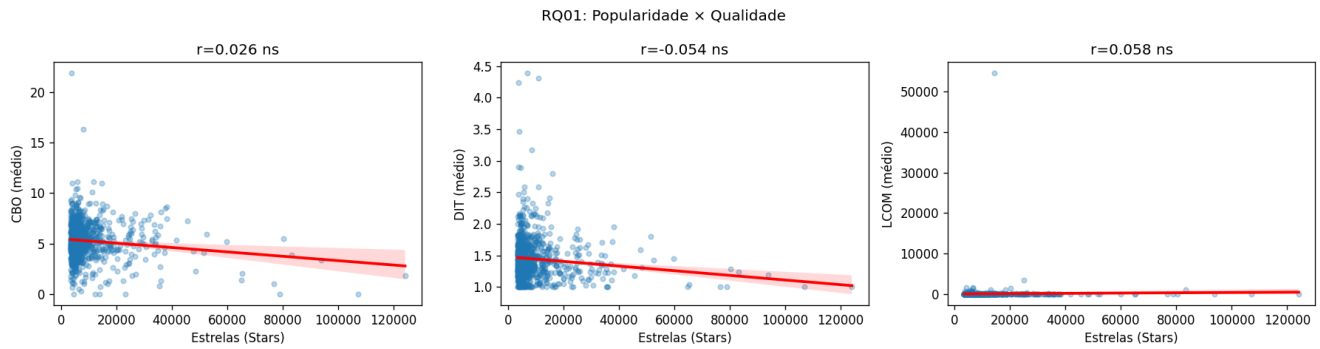


Figura 1: RQ01 — Popularidade × Qualidade

Métrica	r (Spearman)	p-valor	Significância
CBO	0,002	0,949	ns
DIT	0,281	4,3e−19	***
LCOM	0,194	1,0e−09	***

Resultado: CBO sem correlação; DIT e LCOM com correlação positiva significativa. Hipótese **parcialmente refutada** — o efeito existe, mas na direção oposta à qualidade superior.

Interpretação: sistemas mais antigos tendem a ter hierarquias de herança mais profundas e menor coesão, sugerindo acúmulo de débito técnico ao longo do tempo.

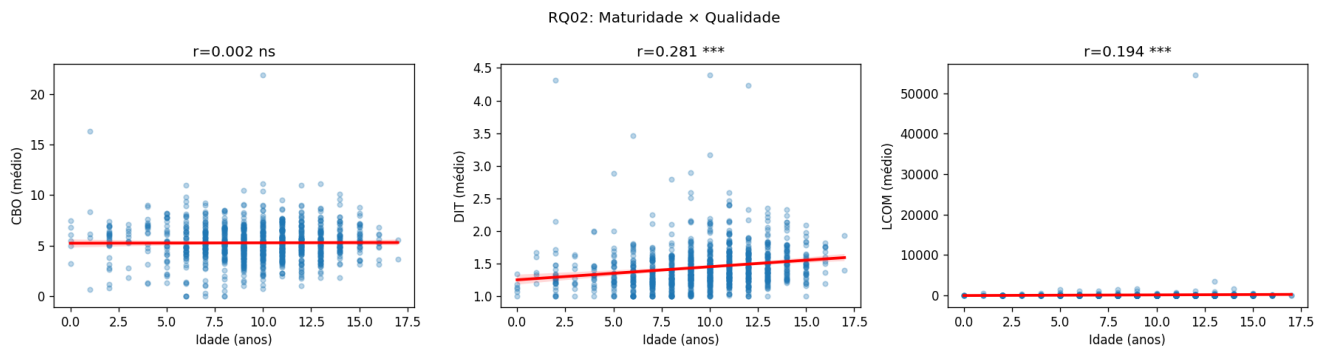


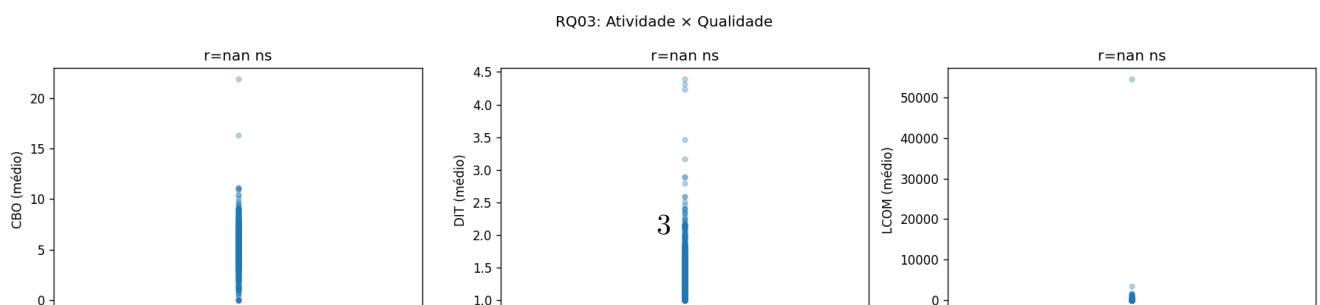
Figura 2: RQ02 — Maturidade × Qualidade

1.4.4 RQ03 — Atividade × Qualidade

Hipótese: correlação negativa entre releases e CBO/LCOM.

Resultado: análise inviabilizada — ReleasesCount é **constante (zero)** em todo o dataset. A REST Search API não retorna o campo de releases; todos os valores foram registrados como 0.

Interpretação: RQ03 **não pôde ser respondida** com os dados atuais. Para investigar, seria necessário enriquecer o dataset via endpoint `/repos/{owner}/{repo}/releases`.



Métrica	r (Spearman)	p-valor	Significância
CBO	0,389	2,5e-36	***
DIT	0,217	9,3e-12	***
LCOM	0,447	1,0e-48	***

Resultado: correlação positiva significativa em todas as métricas. Hipótese **confirmada**.

Interpretação: projetos maiores (mais linhas de código Java) apresentam maior acoplamento e menor coesão. O efeito é especialmente forte em LCOM (r=0,45), indicando que escala tende a dispersar responsabilidades entre classes.

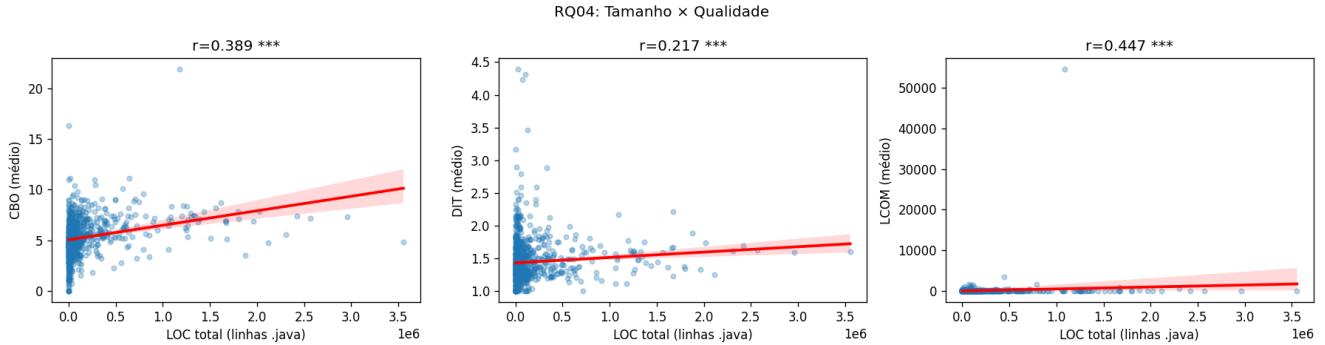


Figura 4: RQ04 — Tamanho × Qualidade

1.5 5. Discussão

RQ	Hipótese	Resultado	Confirmada?
RQ01	Stars vs CBO/LCOM negativamente	Sem correlação	Não
RQ02	Efeito ambíguo em maturidade	DIT e LCOM positivos (débito técnico)	Parcial
RQ03	Releases vs CBO/LCOM negativamente	Inviabilizada (dados zerados)	—
RQ04	LOC vs CBO/LCOM positivamente	Confirmada, forte	Sim

Limitações e ameaças à validade: - **31/1.000 repositórios** excluídos por falhas do CK em construções Java modernas (records, switch expressions, Java 17+) — ameaça à validade interna. - **ReleasesCount** zerado impede resposta de RQ03 — ameaça à validade de construto. - Correlação de Spearman mede associação monotônica, não causalidade. - Métricas CK são calculadas a nível de classe e agregadas pela média — outliers afetam AvgLcom (DP=1.757). - Dataset restrito aos top-1.000 por estrelas — não representa projetos privados ou menos populares.

1.6 6. Conclusão

- **Popularidade não implica qualidade (RQ01):** estrelas refletem alcance, não arquitetura interna.

- **Maturidade está associada a mais débito técnico** (RQ02): sistemas antigos acumulam herança profunda e menor coesão.
 - **Tamanho é o preditor mais forte de qualidade** (RQ04): LCOM e CBO crescem com o volume de código.
 - **Atividade via releases não pôde ser avaliada** (RQ03): limitação de dados da API.
-

1.7 7. Reprodutibilidade

1. Coletar 1.000 repositórios Java

```
dotnet run --project src/MetricsCollector/MetricsCollector.csproj -- --collect-only
```

2. Calcular métricas CK (requer CK_JAR e Java)

```
dotnet run --project src/MetricsCollector/MetricsCollector.csproj -- --ck-only --ck-all
```

3. Calcular LOC

```
dotnet run --project src/MetricsCollector/MetricsCollector.csproj -- --loc-only --loc-all
```

4. Gerar análise e figuras

```
cd analysis && python3 analyze.py
```

Configuração utilizada: LAB02_CK_PARALLEL=8, LAB02_JVM_MAX_HEAP=1g.

Resultado final: data/repositorios_processo.csv (969 linhas com CK, 991 com LOC).